

Axel Ricardo Diaz Mendoza 379652

20/02/26

¿Qué es Markdown?

Markdown es un lenguaje de marcado ligero usado para dar formato a texto plano. Permite crear documentos con estructura (títulos, listas, enlaces, código, etc.) sin usar editores complejos.

Se usa mucho en:

- README de GitHub
- Documentación técnica
- Blogs
- Notas (Obsidian, Notion, etc.)
- Chats y foros

Su ventaja: fácil de leer incluso sin renderizar.

¿Como se utiliza?

Escribes texto normal y agregas símbolos simples para indicar formato.

SINTAXIS

1. Esto es un encabezado H1

1.1. Esto es un encabezado H2

1.1.1. Esto es un encabezado H3

1.1.1.1. Esto es un encabezado H4

1.1.1.1.1. Esto es un encabezado H5

Esto es un texto en *italicas*

Esto es un texto en italicas

Esto es un texto en *italicas*

Esto es un texto en italicas

Esto es un texto en **negritas**

Esto es un texto en negritas

Esto es un texto en **negritas**

Esto es un texto en negritas

Este es un texto que puede ser código

Este es un texto tachado

~~texto tachado~~

- Elemento 1
 - Elemento 2
 - Elemento 3
 - Elemento 3.1
 - Elemento 3.2
 - Elemento 3.2.1
 - Elemento 4
1. Elemento 1
 2. Elemento 2
 3. Elemento 3
 1. Elemento 3.1
 2. Elemento 3.2
 4. Elemento 4

[google.com](#) "Enlace a google"



Productos	Precio	Cantidad
papa	18	1

Esto es una nota

- ☒ Primera tarea
- ☐ Segunda tarea

- ☒ Tercera tarea
- ☐ Cuarta tarea

```
console.log('hello world')
```

```
int numero;

printf("Dame un numero");
scanf("%d", &numero);
printf("El numero es %d", numero);
```

@darhroockie 👍 😊

¿Que es git?

Git es un sistema de control de versiones distribuido, diseñado por Linus Torvalds en 2005 para gestionar el código fuente de proyectos, garantizando velocidad, integridad y flexibilidad. Permite rastrear cambios, colaborar simultáneamente y mantener un historial completo de archivos sin necesidad de una conexión constante al servidor.

¿Que es github?

Una plataforma en la nube basada en Git que funciona como un sistema de control de versiones y alojamiento de repositorios de código. Permite a desarrolladores almacenar proyectos, gestionar cambios en el código a través del tiempo y colaborar con otros usuarios en tiempo real, facilitando la creación de software de manera

¿Como se usa?

1. En tu carpeta de proyecto, inicializas Git
2. Agregas archivos al "stage"
3. Hacer un commit (guardas un punto de control)
4. Conectas tu repo local con github (remote)
5. Haces push para subirlo a la nube

Comandos esenciales de Git

Ver estado y cambios

```
git status
git diff
git log --oneline
```

Configurar tu nombre y correo (una vez)

```
git config --global user.name "Tu Nombre"
git config --global user.email "tucorreo@ejemplo.com"
```

Iniciar repositorio en una carpeta

```
git init
```

Agregar archivos al stage

```
git add archivo.txt
git add .
```

Hacer commit

```
git commit -m "Mensaje claro del cambio"
```

Descargar cambios / actualizarte

```
git pull
```

¿Cómo crear un repositorio en GitHub?

1. En GitHub: New repository
2. Ponle nombre (ej. PORTAFOLIO)
3. Elige Public o Private
4. Marcamos en no agregar README
5. Crea el repo

Eso crea un repositorio vacío

Subir tu proyecto local a GitHub

siya tienes una carpeta con tu proyecto y quieres subirla

En la terminal dentro de la carpeta del proyecto:

```
git init
git add .
git commit -m "Primer commit"
```

Ahora conectas con GitHub (remote). Copia la URL del repo en GitHub (SSH)

pegas el link de ssh con el siguiente comando:

```
git remote add origin git@github.com:USUARIO/REPO.git
```

y subes

```
git branch -M main
git push -u origin main
```

¿Qué es Hugo?

Hugo es un generador de sitios estáticos (SSG).

Convierte archivos Markdown → HTML → Sitio web listo.

✓ Ventajas:

- Muy rápido
- No necesita base de datos
- Ideal para blogs / documentación
- Perfecto para GitHub Pages

¿Qué es GitHub Actions?

GitHub Actions es un sistema de automatización (CI/CD).

Permite:

- Compilar Hugo automáticamente
- Publicar el sitio
- Ejecutar tareas al hacer push
-

Crear un sitio estático con Hugo

Instalar Hugo

Ubuntu / Debian:

```
sudo apt update  
sudo apt install hugo
```

Verificar:

```
hugo version
```

Ejecutar estos comandos para crear un proyecto Hugo con el tema Ananke

```
hugo new site docs  
cd docs  
git init  
git submodule add https://github.com/theNewDynamic/gohugo-theme-ananke.git  
themes/ananke  
echo "theme = 'ananke'" >> hugo.toml  
hugo server
```

ahi ya estaria hecho la pagina hugo

para agregar contenido dentro de hugo usaras el siguiente codigo (en este caso de un ejercicio)

```
hugo new content content/practica0/index.md
```

ya para guardar dentro del repositorio los cambios seria

```
git add .  
git commit -m "ej. actualizar"  
git push
```

y estaria actualizado dentro del repositorio.

para crear un archivo dentro de portafolio para ignorar archivos seria:

```
touch .gitignore  
touch .gitmodules
```

Agregar pagina estatica dentro de pages de github

1. entrar al repositorio
2. ir a ajustes
3. entrar a pages
4. en la parte de build and deployment (seure) seleccionar github accions

ya teniendo eso desde gitbash se crea una nueva carpeta dentro de la carpeta usando los siguientes comandos:

```
mkdir -p .github/workflows
touch .github/workflows/hugo.yaml
```

se copia y pega el siguiente workflow dentro de hugo yaml que es lo que se va a ejecutar cuando queremos crear una pagina

```
name: Deploy Hugo site to pages
on:
  push:
    branches:
      - master
  workflow_dispatch:
permissions:
  contents: read
  pages: write
  id-token: write
concurrency:
  group: pages
  cancel-in-progress: false
defaults:
  run:
    shell: bash
jobs:
  build:
    runs-on: ubuntu-latest
    env:
      DART_SASS_VERSION: 1.97.3
      GO_VERSION: 1.26.0
      HUGO_VERSION: 0.156.0
      NODE_VERSION: 24.13.1
      TZ: Europe/Oslo
    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          submodules: recursive
          fetch-depth: 0
      - name: Setup Go
```

```

    uses: actions/setup-go@v6
    with:
      go-version: ${{ env.GO_VERSION }}
      cache: false
  - name: Setup Node.js
    uses: actions/setup-node@v6
    with:
      node-version: ${{ env.NODE_VERSION }}
  - name: Setup Pages
    id: pages
    uses: actions/configure-pages@v5
  - name: Create directory for user-specific executable files
    run: |
      mkdir -p "${HOME}/.local"
  - name: Install Dart Sass
    run: |
      curl -sLJO "https://github.com/sass/dart-
sass/releases/download/${DART_SASS_VERSION}/dart-sass-${DART_SASS_VERSION}-linux-
x64.tar.gz"
      tar -C "${HOME}/.local" -xf "dart-sass-${DART_SASS_VERSION}-linux-
x64.tar.gz"
      rm "dart-sass-${DART_SASS_VERSION}-linux-x64.tar.gz"
      echo "${HOME}/.local/dart-sass" >> "${GITHUB_PATH}"
  - name: Install Hugo
    run: |
      curl -sLJO
"https://github.com/gohugoio/hugo/releases/download/v${HUGO_VERSION}/hugo_extended
_${HUGO_VERSION}_linux-amd64.tar.gz"
      mkdir "${HOME}/.local/hugo"
      tar -C "${HOME}/.local/hugo" -xf "hugo_extended_${HUGO_VERSION}_linux-
amd64.tar.gz"
      rm "hugo_extended_${HUGO_VERSION}_linux-amd64.tar.gz"
      echo "${HOME}/.local/hugo" >> "${GITHUB_PATH}"
  - name: Verify installations
    run: |
      echo "Dart Sass: $(sass --version)"
      echo "Go: $(go version)"
      echo "Hugo: $(hugo version)"
      echo "Node.js: $(node --version)"
  - name: Install Node.js dependencies
    run: |
      [[ -f package-lock.json || -f npm-shrinkwrap.json ]] && npm ci || true
  - name: Configure Git
    run: |
      git config core.quotepath false
  - name: Cache restore
    id: cache-restore
    uses: actions/cache/restore@v5
    with:
      path: ${{ runner.temp }}/hugo_cache
      key: hugo-${{ github.run_id }}
      restore-keys:
        hugo-
  - name: Build the site

```

```

      run: |
        cd docs
        hugo build \
          --gc \
          --minify \
          --baseURL "${{ steps.pages.outputs.base_url }}" \
          --cacheDir "${{ runner.temp }}/hugo_cache"
    - name: Cache save
      id: cache-save
      uses: actions/cache/save@v5
      with:
        path: "${{ runner.temp }}/hugo_cache"
        key: "${{ steps.cache-restore.outputs.cache-primary-key }}"
    - name: Upload artifact
      uses: actions/upload-pages-artifact@v3
      with:
        path: ./docs/public
  deploy:
    environment:
      name: github-pages
      url: "${{ steps.deployment.outputs.page_url }}"
    runs-on: ubuntu-latest
    needs: build
    steps:
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4

```

ya teniendo este codigo dentro del workflow guardamos los cambios

```

git add .
git commit -m "Se creo un archivo hugo.yaml"
git push

```

y ya con eso tendrias la pagina estatica dentro de actions de github

ya cada que hagas un guardar cambios en accion se grabarian los procesos que hiciste

Enlace de mi repositorio

<https://github.com/Wiizpy/PortafolioPP>

Enlace de mi pagina estatica

<https://wiizpy.github.io/PortafolioPP/>